Mehran University Research Journal of Engineering and Technology
Vol. 41, No. 1, 65 - 79, January 2022
p-ISSN: 0254-7821, e-ISSN: 2413-7219
DOI: https://doi.org/10.22581/muet1982.2201.07

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

Resume Classification System using Natural Language Processing and Machine Learning Techniques

Irfan Ali[1a], Nimra Mughal[1b], Zahid Hussain Khand[1c], Javed Ahmed[1d], Ghulam Mujtaba[1e]

RECEIVED ON 02.11.2020, ACCEPTED ON 05.05.2021

ABSTRACT

The selection of a suitable job applicant from the pool of thousands applications is often daunting job for an employer. The categorization of job applications submitted in form of Resumes against available vacancy(s) takes significant time and efforts of an employer. Thus, Resume Classification System (RCS) using the Natural Language Processing (NLP) and Machine Learning (ML) techniques could automate this tedious process. Moreover, the automation of this process can significantly expedite and transparent the applicants' screening process with mere human involvement. This experimental study presents an automated NLP and ML-based RCS that classifies the Resumes according to job categories with performance guarantees. This study employs various ML algorithms and NLP techniques to measure the accuracy of RCS and proposes a solution with better accuracy and reliability in different settings. To demonstrate the significance of NLP and ML techniques for RCS, the extracted features were evaluated on nine ML classification models namely Support Vector Machine - SVM (Linear, SGD, SVC and NuSVC), Naïve Bayes (Bernoulli, Multinomial & Gaussian), K-Nearest Neighbor (KNN), and Logistic Regression (LR). The Term-Frequency-Inverse-Document-Frequency (TF-IDF) feature representation scheme was proved suitable for RCS. The developed models were evaluated using the Confusion Matrix, F-Score, Recall, Precision, and overall Accuracy. The experimental results indicate that using the One-Vs-Rest-Classification strategy for this multi-class Resume classification task, the SVM class of Machine Learning classifiers performed better on the study dataset of over nine hundred sixty plus parsed resumes with more than 96% accuracy. The promising results suggest that NLP and ML techniques employed in this study could be used for developing an efficient RCS.

Keywords: Resume Classification, Natural Language Processing, Machine Learning, Text Classification, Recommender System

1. INTRODUCTION

Internet-based recruiting systems have been rapidly adopted by recruiters in recent years. The rapid growth of the internet caused an identical growth in quantity of obtainable online information [1]. As a result, information is widely available. Contrary to this, information became overloaded and resulted in the need for information management [2, 3]. Moreover, the ever increasing unemployment rate in developing countries like Pakistan results in considerable amount of job applications for a vacant position [4]. Thus, the selection of suitable job applicants from the pool of thousands of applications is often a daunting job for an employer. Recruiters need to screen through a large amount of data to select the most suitable application from the pool. Thus, it significantly increases the workload of the concerned department of Recruiter [5]. Moreover, this process involves the engagement of considerable Human Resources and requires rigorous efforts and resources to finalize the most suitable applicant for further

[1] Center of Excellence for Robotics, Artificial Intelligence, and Blockchain, Department of Computer Science, Sukkur IBA University, Sukkur-65200, Sindh Pakistan. Email: [a]irfan.memon@iba-suk.edu.pk (Corresponding Author), [b]nimar.mscsf19@iba-suk.edu.pk, [c]zahid@iba-suk.edu.pk, [d]javed@iba-suk.edu.pk, mujtaba@iba-suk.edu.pk.

This is an open access article published by Mehran University of Engineering and Technology, Jamshoro under CC BY 4.0 International License.

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

recruiting process. If the recruiters can figure out the non-relevant profiles at the earlier stages of the hiring process, this can significantly save time and money [6].

The Resume is a portfolio document developed by job applicants to present the relevant details for the vacant job. In this document, the applicant provides personal details, Educational details, accomplishments, competencies, skills, and experiences. This resume helps recruiters to shortlist the applicant from the pool of applications as it provides the complete picture of the applicant's competencies and skills. The resume screening demands domain knowledge to understand the suitability and relevance of an applicant for the advertised job vacancy. However, the current global economic condition that companies face of getting less capital to speculate within their HR department, while desperate to ensure that they are choosing the highly competitive applicant fitted to the job description [1]. Thus, recruiters are facing three main challenges:

- **Making sense of Resume:** This is a fact that Resumes in the market have no defined standard. Every resume may have a different structure in the pool of applications. Thus, HR needs to manually go through each resume to find out the best resume.
- **Mapping resume to the job description:** This is based on mapping the applicant's Resume to the requirements criteria provided by the recruiter. This process involves detailed screening and requires domain experts to efficiently perform this task.
- **Managing the cost:** For, Screening and selection, Recruiters need to adopt automated processes with mere human involvement to save time and money.

Hence, Machine Learning based automated Resume Classification Systems can be used to classify the Resumes according to the job category. This approach can automate the tedious process of Resume Selection and support recruiters to overcome the above-mentioned challenges. Moreover, the automation of this process can significantly expedite the applicants' shortlisting process and transparent the selection process with mere human involvement.

Text Classification (TC) is a technique to automatically classify the predefined classes relevant to a particular text document [7, 8]. TC is one of the most fundamental tasks of Natural Language Processing (NLP). TC is carried out with the involvement of Supervised Machine Learning techniques. These techniques require text representation as a fixed-length feature vector [7]. Thus, Preprocessing and Feature Engineering are the most important and fundamental steps for such text classification tasks where we apply various feature extraction and feature representation techniques [9].

Feature extraction typically finds the set of most informative features whereas feature representation figures out the most suitable way to represent the values of extracted features. The most widely used feature extraction techniques for text documents are N-grams, Bag of Words (BoW), and Word-to-Vec. Every extracted feature assigned the numeric value using different representation techniques such as *Binary* and *TF-IDF*. Every feature engineering task has some pros and cons. Hence, the job of a Machine Learning Engineer is to find the most useful technique for the problem under consideration. Nevertheless, various Machine Learning approaches have been proposed to develop Resume Classification Systems in literature. However, this study aims at developing an ML-based system that classifies the Resumes according to job categories. The study applies the Supervised Machine Learning approach for resume classification to correctly classify 25 different job categories resumes belong to. The dataset has 962 labeled resumes' categories to train the classifier. Thus, various multi-class classification algorithms and NLP techniques are employed to measure the accuracy of Resume Classification using performance metrics such as overall accuracy, F-Score, Precision, and, Recall. This study proposes an ML-based Resume Classifier with better accuracy and performance guarantees.

The resume is an official and formal document used mainly for demonstrating the brief profile of a job applicant. The resume contains information related education, skills, experience, achievements, and portfolio of a job applicant. The resume often used as an effective tool to assess the overall suitability of an

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

aspirant for the desired job. Moreover, in response to job postings applicants submit Resume as a formal document for job application consideration. The employer receives hundreds of Resumes for mere vacancies and finds it difficult to categorize and classify to a suitable job vacancy. Thus, this study attempts at developing an efficient and accurate Resume Classification System to ease the job of employers.

The study proposes an automated Resume Classification System (RCS) using state-of-the-art Natural Language Processing (NLP) techniques for processing Resumes (plain-text document submitted as an job application) and Machine Learning (ML) algorithms (classifiers) for classification of Resumes as per available job category. The major contribution of study lies in preprocessing the Resumes to corpus and vectorized representation using the NLP techniques suitable for classification task carried out by ML-based algorithms and classifiers. Moreover, the experimental evaluation of various features extraction and representation schemes are major contribution to body of knowledge for Resume Classification task. In addition, this study presents experimental performance comparison of various ML-based classifiers for the Resume or plain-text classification task. This study can serve as basic building block for developing an automated, robust, and reliable RCS that could be employed to in real-time application of applicants shortlisting process based on Resumes.

The rest of the paper is organized as follows. Section 2 presents the review of related studies. Section 3 describes the proposed methodology to accomplish the objectives, Section 4 presents and discusses the findings of the study, and Section 5 presents the major limitations of the study and proposed future work and finally, Section 6 concludes the study.

2. RELATED WORK

In recent years, the ML based Text Classification (TC) techniques have been widely employed in various domains [10] such as Sentiment analysis [11, 12], E-Commerce portals [13, 14], Email classification [15], Human Resource Management [2], Banking and Exchange Stock Markets [16, 17], and bioinformatics [18, 19]. In this study, ML-based text classification techniques are employed in the Human Resource Management domain. Various NLP and ML classification techniques have been employed to predict the category of Resume.

Several studies have proposed the Machine Learning based system for Human Resource Management and recruiting processes. For instance, the study [20] designed the approach for Resume ranking that uses that layered information retrieval framework to parse the resumes. The goal of this study was to help recruiters to find out the relevant job applicant for a job opening. Another study [21] designed the personalized approach for Resume-job matching that offers the statistical similarity for resume ranking according to the available jobs. This study could have been more generalized to recruiters as well as for job seekers. Employers can make use of this system to find the relevant resumes whereas job seekers can use to search the most relevant job matches their resumes. The fuzzy-based model used in [22] to evaluate the relevancy of a resume as compared to the job description. All the above-mentioned studies are working for document similarity by comparing the resume to the job description. However, few studies employed Supervised Text Classification Techniques to predict the category of Resume.

Perhaps, the most related work to the proposed approach is of [23]. In this wok, NLP and ML techniques were employed to predict the domain of resumes. This study aimed to allocate the relevant project to recruits. The study proposed the Named Entity Recognition (NER) approach coupled with various classification models such as Logistic Regression, K-Nearest Neighbors for the classification. Besides this, the study proposed an ensemble learning-based voting classifier that was retrained after a fixed interval. Hence, the number of votes for each classifier was modified. The experimental results revealed that a voting based classifier produced 91.2% accuracy in predicting the categories while the accuracy was 84.2% without retraining. Another related study is of [24], in which the Convolutional Neural Network (CNN) was used to classify Resumes into 27 different job categories. In

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

this study, CNN classifier was trained on word2Vec pre-trained representations to determine the category of Resume. This approach achieved 40.15% accuracy on resume classification and 74.88% accuracy on the job classification task. However, the study only used job summary text for classification and considered only one base method of fast Text for comparison of the performance.

Hence, both the aforementioned studies had some major limitations. The aforementioned studies had employed various classification techniques whereas failed to evaluate various preprocessing techniques for the proposed classifiers which may lead to low accuracy of the classifier. Further, only overall accuracy as a measure used for evaluation and failed to use various evaluation metrics such as F-Score, Precision, and Recall to evaluate the learning efficiency of classifiers.

It is evident from the above mentioned studies that approaches used mainly suffered with two problems lower accuracy and performance comparison. Besides this, very few ML models were employed for the Resume Classification task and accuracy as the only measure used for performance. Moreover, the features extraction and representation techniques were not explored to overcome the less accuracy problem. To overcome the limitations of previously proposed studies, this study will use different NLP and Machine Learning techniques to improve the efficiency of classifiers and various performance matrices will be used for model evaluation. Also, various feature extraction and representation techniques would be employed for discriminative features contributing to better classification. Further, this study will provide discriminative features to several machine learning models, and various performance matrices such as $Precision_M$, Recall, and F-Score will be used for performance measuring.

3. METHODOLOGY

This Section discusses the proposed methodology for building an efficient and accurate Resume Classification System in detail. To achieve the objective of Resume Classification, NLP and ML techniques are employed using the best practices. The overall methodology is divided in five stages as illustrated in Fig. 1: (i) Data Collection and visualization (ii) Preprocessing (iii) Feature Engineering (iv) Model Construction and (v) Model Evaluation and testing in a real-time environment using Graphical User Interface (GUI).

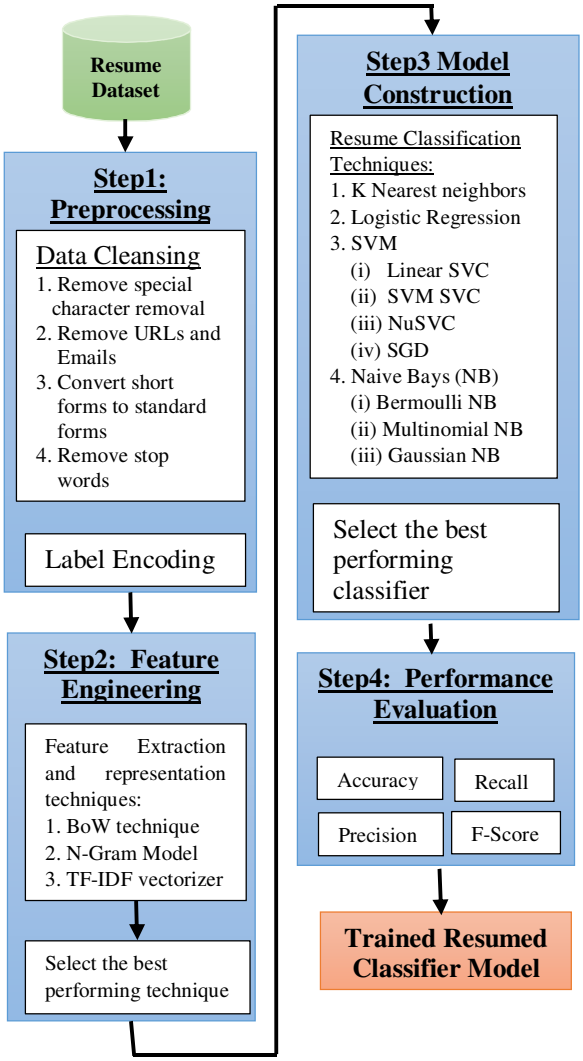


Fig. 1: The proposed methodology for resume classification

3.1 Data Collection and Visualization

The Resumes with Job Categories dataset were collected from an online data repository. The dataset is in Comma Separated Values (CSV) file format and has three columns namely ID, Category, and Resume's Text. The ID, Category and Resume Columns represent Index, Job Category/Field, and content of the resume respectively. The dataset contains 962 parsed and labelled resumes in 25 different job categories.

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

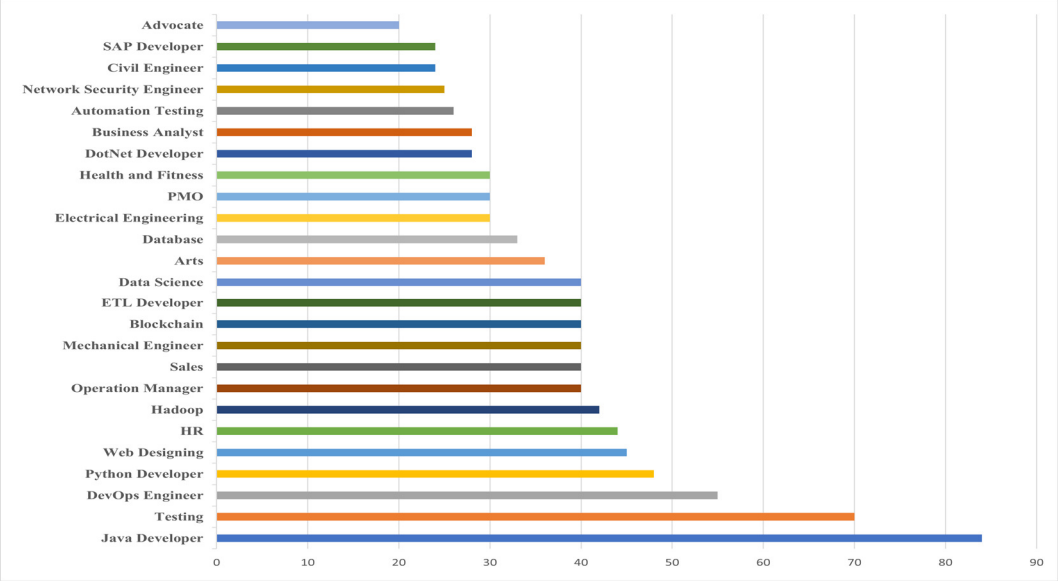


Fig. 2: Resume instances for each job category

The number of resume instances for each class job category is illustrated in Fig. 2 for more appealing representation, Table 1 for clear class distribution, and Fig.3 for category-wise distribution (percentage) of resume instances plotted using Python Matplotlib library. The visual evidence in Fig. 2 shows that each job category has a different number of resume instances and this can lead to an imbalanced data problem.

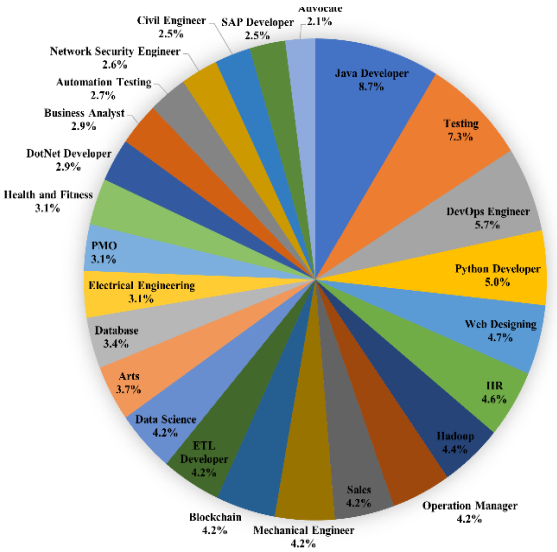


Fig. 3: Category-wise total distribution of Resume instances

Table 1: Resume Instance for each Category	
Java Developer	81
Testing	70
Development Engineering	55
Python Developer	48
Web Designing	45
HR	44
Hadoop	42
Operation Manager	40
Sales	40
Mechanical Engineer	40
Blockchain	40
ETL Developer	40
Data Science	40
Arts	36
Database	33
Electrical Engineering	30
PMO	30
Health and Fitness	30
DotNet Developer	28
Business Analyst	28
Automation Testing	26
Network Security Engineer	25
Civil Engineer	24
SAP Developer	24
Advocate	20

Moreover, the data for two categories namely *Java Developer* and *Testing* has the highest resume instances and can be considered as biased class categories. Whereas, the resume instances for some

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

categories such as *Advocate*, *Civil Engineer*, and *SAP Developer* are relatively less than some other categories for instance *Java Developer*. However, the category-wise total distribution in Fig. 3 illustrates the overall representation of resume instances within the percentage range of (2.1 to 8.7%) in the dataset. This illustration provides an intuition that at some extant dataset has class imbalance problem. However, the classification accuracy is not affected with this imbalance class distribution due to employed effective feature extraction methodology for classification task.

3.2 Data Preprocessing

The Data preprocessing involves steps to transform raw data into meaningful information for the Machine Learning task. In the case of textual data for text classification, these steps involve cleaning raw text data, removing the unnecessary or meaning-less data, removing the repetitive (redundant) data, removing the missing (null) values, and transforming data to a common scale. To preprocess the resume's textual data for the Resume Classification task following key steps were performed.

3.2.1 Data Cleansing

The dataset contains the parsed resumes from different formats such as PDF, DOC, DOCX in a CSV format. It has a lot of unnecessary and unprocessed data in the resume column. Thus, the major efforts were required to preprocess the data and make it ready for Text Classification. In the data preprocessing step, the less informative text was cleaned using the Natural Language Processing Took Kit - NLTK [23] for stop words removal and Python 3.7.3 Regular Expressions. The following key tasks were performed for data preprocessing using the customized written program function in Python.

i) The textual content of resumes was converted to lowercase.
ii) The special characters, punctuations, brackets, URLs, Email addresses, mentions, hash tags, apostrophes, leading and trailing characters, extra white spaces, and Non-ASCII characters were removed from the Resume's text.
iii) The masking was applied to special escape sequences such as \n, \t, \a, \b, and so on.
iv) The numbers were masked.
v) The string fragmentations were masked.
vi) The word phrases in short form such as I'll to I will were converted to their full forms.
vii) Similar attributions were performed on unclean/unprocessed raw resume's text data.

3.2.2 Removal of the stop words

Stop words removal is one of the most essential steps in data preprocessing. Stop words such as 'is', 'each', 'and' and so on appear most often in any textual data. However, these most frequently occurring words in a text document are not the informative features (tokens) for any classifier. Thus, these stop words should be removed from the corpus for the classification model. The stop words from the resume's text column were removed by performing the following steps using the Python programming:

i) The word tokenization was performed on the resume's text using NLTK library and tokens were stored in an array.
ii) The standard English language stop words were imported using NLTK corpus and compared with each element in the tokenized array.
iii) If any element of the tokenized array was found in the list of NLTK stop words, that particular element (tokenized word) was removed.
iv) Repeated this process for all the tokens. The final tokenized elements array did not contain any stop word

To visualize the stop words removal process, the word cloud of most frequently occurring words in the corpus of resumes was generated using the Python word cloud feature as illustrated in Fig. 4. It can be observed that



Fig. 4: Word Cloud of most frequent words in the cleaned dataset

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

the word cloud now contains more informative words other than frequently occurring stop words and these words would be more meaningful for classifiers to learn.

3.2.3 Stemming and Lemmatization

Stemming and Lemmatization are known as Text Normalization or sometimes Word Normalization techniques in Natural Language Processing (NLP). The purpose of these techniques is to decrease word inflection in the corpus of classification text by mapping the group of words to the same *root stem*. Specifically, stemming and lemmatization remove the prefixes and suffixes (affixes) such as (-es, -s, -ed, in-, un-, -ing, *etc*) from words which result in inflectional (changing/deriving meaning of words). For instance, the *stem (root)* word for *Plays, Playing, and Played* is *Play* so the *stemming and lemmatization* techniques would map these words in the corpus of classification text to *root (stem)* word. Using the above mappings, a sentence could be normalized using the stemming and lemmatization techniques as follow:

The Natural Language Tool Kit (NLTK) library in Python offers the implementation of stemming and lemmatization techniques with different settings. However, unlike stemming offered by the NLTK library in Python, the lemmatization reduces the inflected words properly by ensuring the root word belongs to the language. Thus, lemmatization us applied on Resume's text corpus for text normalization as Resumes are more formal document and lemmatization ensures the proper word structure in normalized text. In our implementation lemmatization text normalization technique produced promising results for corpus tokenization and vectorization.

3.2.4 Label Encoding

The label encoding technique handles the categorical values of variables in the Machine Learning Model. The label encoding technique assigns a unique integer value to a categorical variable. To make raw text data ready for the machine learning model the label encoding was done to assign a numerical label to all categories, as shown in Fig.2. The Scikit-learn Label Encoder was used for the mentioned purpose. Hence, the label encoder on the Category field of the data was applied.

3.3 Feature Engineering

The feature engineering helps to extract, formulate, and represent the set of most discriminative (informative) features from the corpus of text for the classification task. After data cleaning and preprocessing, the resume's corpus has informative set of words as depicted in word cloud in Fig. 4. The Fig. 4 shows that the dataset does not contain stop words and other less informative words. The feature engineering process in Machine Learning mainly involves feature extraction and representation techniques for classification task. Therefore, different feature extraction techniques; for word and character vectorization with varying range of hyperparameters were compared as discussed in Section 3.3.1. For feature representation, different variants of Term-Frequency-Inverse-Document-Frequency (TF-IDF) suitable document representation scheme for plain-text classification [26, 27] was evaluated as discussed in Section 3.3.2.

3.3.1 Feature Extraction and Master Feature Creation

After applying the preprocessing step on the data, the dataset contains the words that are important features for the classification. To demonstrate the significance, different variants for feature extraction namely, *BoW*, *Word Vectorizer*, and *Character Vectorizer* with varying ranges of n-grams were evaluated. However, proposed model yielded better accuracy on *Word Vectorizer* implementation using the TF-IDF feature representation scheme.

In features extraction techniques, word vectorization with different range of hyperparameters unigram (N=1), bigram (N=2), and n-gram (2-6) [25] were compared. Moreover, character vectorization with hyperparameter n-gram (2-6) was also evaluated. However, word vectorization with hyperparameter n-gram (2-6) and max-features set of 1500 yielded better accuracy and improved training and computation time.

3.3.2 Feature Representation

This step aims to allocate an arithmetic value to each of the extracted features in the vector. Term-

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

Frequency-Inverse-Document-Frequency (TF-IDF) has been reported as better performing feature representation scheme in various plain-text classification studies in the literature [19, 26, 28]. Therefore, TF-IDF [27] was used for representing the value of each extracted feature. TF-IDF is a numerical statistic that is intended to find the importance of a word to a document in text corpus collection. This technique is concerned with two things. TF is concerned with the occurrences of each word/feature and determines how frequently the word appears in each document. Whereas, IDF is used to determine the weight of each word in the document. The objective of TF-IDF feature representation is to weigh down the more frequent words while scaling up the rare words in the document.

Hence, TF-IDF Vectorizer was implemented using Python Scikit-Learn library. It is used to perform both feature extraction and feature Representation for the task. To compare the performance of most discriminative features, different values for the max-feature sub-set were tested. However, the accuracy of classifiers was decreasing as the max-feature value was increased. For instance, the max-feature value 2000 and 1500 resulted in an accuracy of 95% and 97% respectively on SVM-SVC. Thus, it can be concluded that the larger value of the max-feature sub-set was not significantly contributing to better accuracy so the max-feature value was set to 1500.

3.4 Resume Classifier Construction

The discriminative features extracted using the techniques described in the previous section were used to build the classifier to accurately classify the Resumes. Several Machine Learning classifiers were opted to select the best performing model for deployment and Graphical User Interface (GUI). The details of Classifier construction is presented in sections below.

3.4.1 Implementation details and experimental setup

After extracting features from the dataset, the data was divided into training and testing. The dataset was divided into 70% and 30% for train and test set respectively. Nine different text classifiers were employed as each has its own philosophy to classify the instances. The "One-Vs-Rest-Classification" strategy for multiclass classification was used [26]. The brief description of the implemented nine machine learning models is as follows:

1. **K Nearest Neighbors (KNN):** KNN is based on finding k-nearest data points to the new instance and assign the label according to the highest neighboring data points. KNN is also known as a lazy learner classifier because of its simplest method of Euclidian distance equation (1) for classification tasks [26].

$$d(p,q) = \sqrt{\sum_{n=1}^{n}(q_i - p_i)^2} \quad (1)$$

2. **Multinomial Naïve Bayes (MNB):** Naïve Bayes classifier is based on the conditional probability. NB classifier finds the probability of a vector belonging to the class. It finds out the probability for all the given instances and classifies with the conditional probability. It is based on strong independence between the features. MNB is one variant of Naïve Bayes that multinomial distribution of all pairs [27].
3. **Bernoulli Naïve Bayes (BNB):** it is also a variant of Naïve Bayes that accepts the binary features only. BNB is also effective for classification tasks [28].
4. **Gaussian Naïve Bayes (GNB):** It is also a variant of NB that supports continuous-valued features that are assumed to be distributed according to Gaussian distribution. GNB only supports vectorized features representation to implement GNB vectorized features representation used [29].
5. **Logistic Regression (LR):** Logistic Regression applies the logistic function on the classification task with a threshold value. LR is considered one of the easiest implementations for classification problems [30].
6. **Linear Support Vector Classifier (SVC):** It is based on finding the best separating line between two classes. It is the simplest form of Support vector machine that finds the linear hyperplane between two classes. Although, it will not give good results if the data is not linearly separable. Linear SVM is also known as the least square Support Vector Machine classifier [31].

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

7. **Support Vector Classifier (SVC):** SVC overcomes the above-mentioned issue of Linear SVM by using the Kernel concept [32] that works well on data that is not linearly separable.
8. **Nu-Support Vector Classifier (NuSVC):** It is similar to the SVC but it also uses a parameter to control the number of support vectors.
9. **Stochastic Gradient Descent (SGD):** It uses SGD for training (that is, looking for the minima of the loss using SGD).

The extracted features and learned ML models were stored in Python external pkl file format for future evaluation and testing. The scikit-learn externals joblib library was used to store extracted features representation and learned models on disk and later used in GUI for real-time testing.

3.4.2 Graphical User Interface and System Evaluation in a Real Time Environment

To evaluate the trained and learned ML models in real-time settings on unseen data the Graphical User Interface (GUI) is designed using the Python Tkinter (Fig. 5). The extracted features and learned ML models are imported to be used in GUI. The designed and developed GUI allows users to provide a resume in text format or select a resume text from an unseen test dataset. The GUI also leverages users to select from nine ML learned models for classification of the resume. This implementation ensures the transparency and real-time analysis of Resume Classification on nine learned models. The designed GUI would also be helpful for implementing Machine Learning models in a real-time environment and helpful for recruiters to tackle the tedious task of Resume Classification in different job categories.

3.5 Evaluation Matrices

To measure the performance of the mentioned Classification models, we used different performance evaluation matrices. As the dataset was imbalanced (shown in Fig. 2 and 3) so the overall accuracy was not only a significant matrix for model evaluation. Therefore, for performance evaluation, Overall accuracy, Precision, Recall, F-Score matrices were used. The brief description of performance matrices is as follows.

I. **Overall Accuracy:** Accuracy is a fraction of predictions that are correctly identified by the algorithms. However, Accuracy itself does not tell the full story when we are working with the imbalanced data.

II. **Micro Precision:** Precision attempts to answer from all the positive predictions, what fraction of actually positive? The value of precision is between 0 and 1. Any model that does not produce false positive results has a precision of 1. It gives us the idea that how precisely the model is identifying the True positive values of classes. In multiclass classification problem precision for all classes is results has a precision of 1. It gives us the idea that

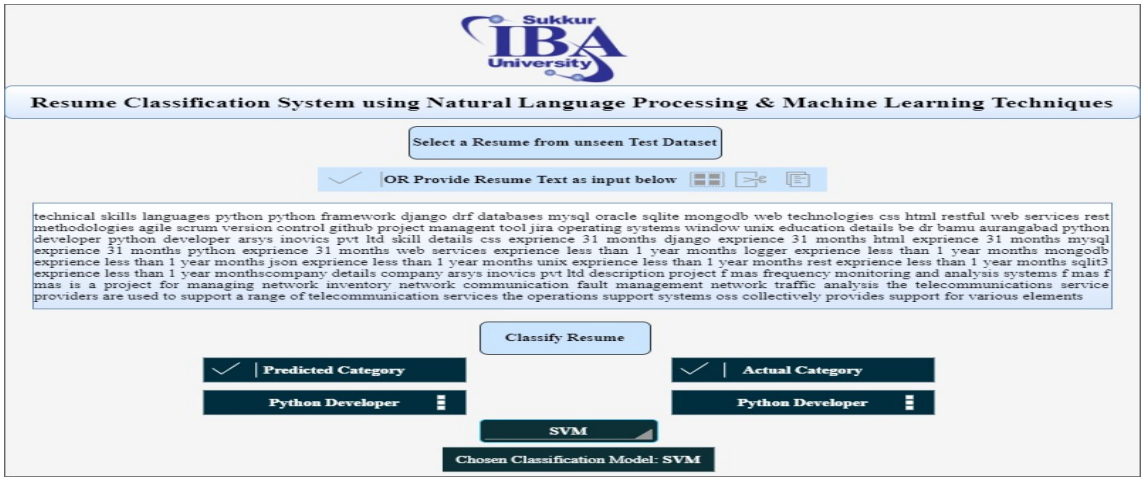


Fig. 5: Graphical User Interface of the proposed system

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

how precisely the model is identifying the True positive values of classes. In multiclass classification problem precision for all classes is computed and then the average of all results is computed. Macro-average computes the metric independently for each class and then takes the average. The mathematics definition of Precision is as follows:

$$\text{Precision} = \frac{\sum_{i=1}^{C} \frac{TP_i}{TP_i + FP_i}}{C}$$

III. **Macro Recall:** Recall attempts to answer that from all the actual positive records, what fraction is correctly identified? The value of precision is between 0 and 1. Any model that does not produce false-negative results has a precision of 1. In multiclass classification problem precision for all classes is computed then the average of all results computed. Macro-average computes the metric independently for each class and then takes the average. The mathematical definition of Recall is as follows.

$$\text{Recall} = \frac{\sum_{i=1}^{C} \frac{TP_i}{TP_i + FN_i}}{C}$$

IV. **Macro F-Score:** F-Measure is defined by the weighted harmonic mean of test's precision and recall. The values are between 0 and 1 where highest value '1' shows that algorithm reaches to best precision and recall values.

4. RESULTS AND DISCUSSION

Table 2 presents the Precision, Recall, F-Score, and overall accuracy of all the trained models on test data. The variation in the performance of trained models can be significantly observed. The Support Vector Machine class of learning algorithms performs better than other classifiers. In all 318 analyses on test data instances, the Linear Support Vector Classifier out-performs the other eight classifiers with nearly 98% overall accuracy and 1.0 precision. It can be generalized that for the Resume Text Classification task, the SVM class classifiers performs best.

Table 2 summarizes the Precision, Recall, F-Score, and overall Accuracy of classifiers on testing data. The results show that most of the algorithms produced excellent results on study data. This can be comprehended as the dataset size was optimal and best NLP and ML techniques were employed to achieve significantly better results. It is also shown that LSVC, SGD, LR, and SVC produced exceptionally well results. Thus, the LSVC classifier is the best performing classifier.

Fig.6 illustrates the overall accuracy and misclassification report of the classifiers. It can also be seen that the Bernoulli Naïve Bayes (BNB) did not produce better results as compared to all other classifiers while the Multinomial Naïve Bayes (MNB) performed well on the dataset. The misclassification of BNB is high as compared to all other classifiers. One of the reasons for that misclassification is Bernoulli's classifier is mainly used for Binary classification and treating all values as the negative class whereas, the Resume Classification is a multi-class problem. Most of the models produced approximately similar results except the BNB.

Table 2: Performance Evaluation of learned ML Models

Classifier	Precision	Recall	F-Score	Overall Accuracy %	Misclassification %
LSVC	1.00	1.00	1.00	99.6	0.4
SGD	1.00	1.00	1.00	99.6	0.4
LR	1.00	0.99	0.99	99.3	0.7
SVC	1.00	0.99	0.99	99.3	0.7
NuSVC	0.99	0.99	0.99	99.3	0.7
KNN	0.99	0.98	0.99	97.2	2.8
GNB	0.98	0.96	0.96	96.5	3.5
MNB	0.98	0.95	0.96	94.8	5.2
BNB	0.89	0.76	0.79	79.2	20.8

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

The overall misclassification report is relatively low, thus this can be inferred that the extracted features using TF-IDF were the most discriminative for the Resume Classification Task. Moreover, the GNB and BNB models require a vectorized representation of features and this could be a reason for slightly poor performance.

Fig. 7 illustrates the Precision, Recall, FScore of all the models. There is a minor difference in the Precision, Recall and F-Score. Well, this was not the case when un-processed data was used. The same performance matrices were measured on raw data and results were not encouraging. Hence, our designed methodology extracted the most discriminative features from the dataset. That is the reason why most of the classifiers yielded the best performance.

Fig. 8 illustrates the Train versus Test accuracy of the used nine classifiers. The overall dataset was divided into 70% and 30% for training and testing respectively. Machine Learning models often suffer with *overfitting* and *underfitting* problems.

The overfitting problem occurs when the learned ML model performs best on *training data* and yields better

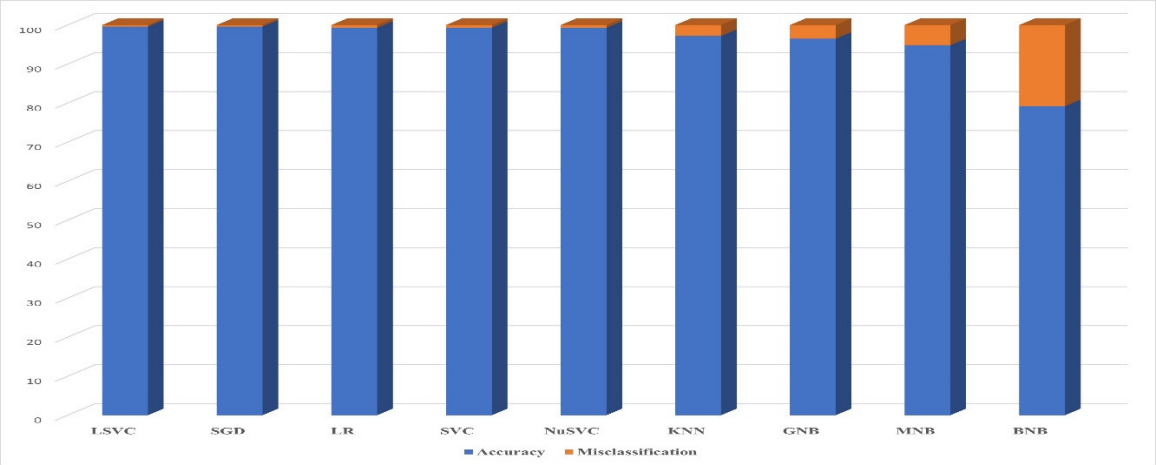


Fig. 6: Overall Accuracy vs Misclassification Report

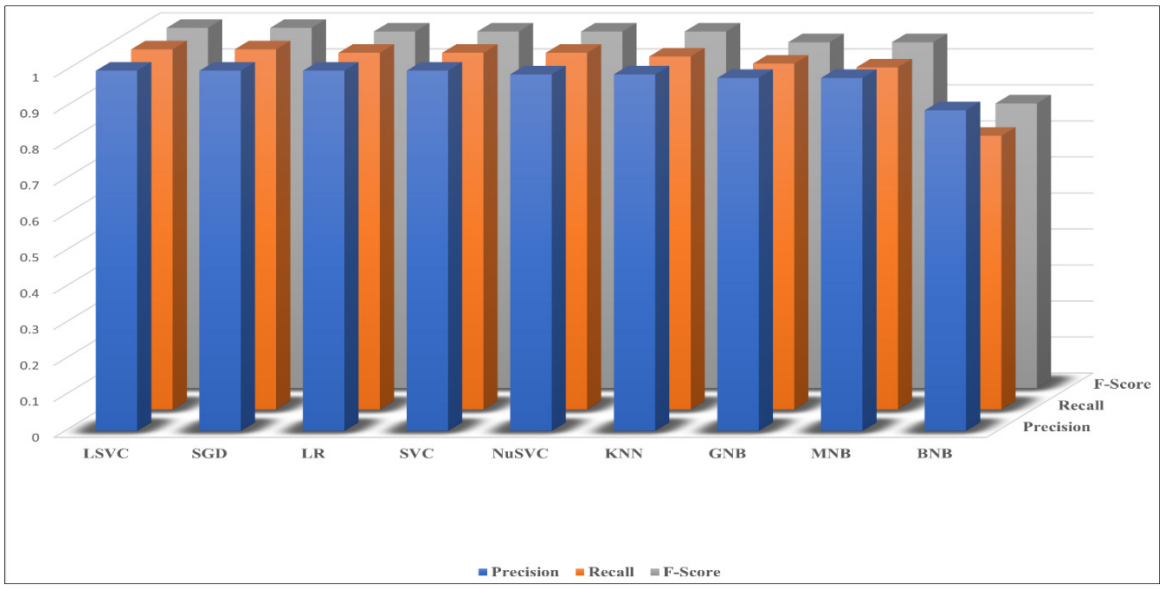


Fig. 7: Precision, Recall, F-Score, - Performance Matrices

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

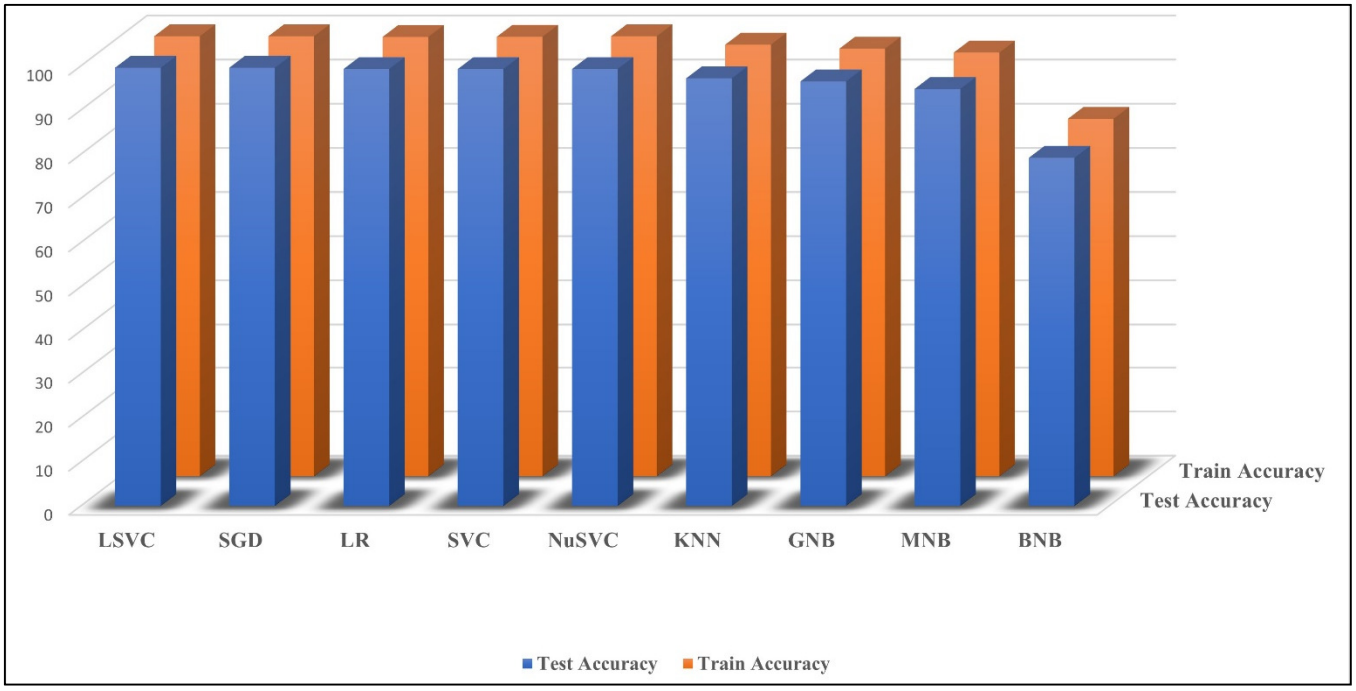


Fig. 8: Train vs Test Accuracy

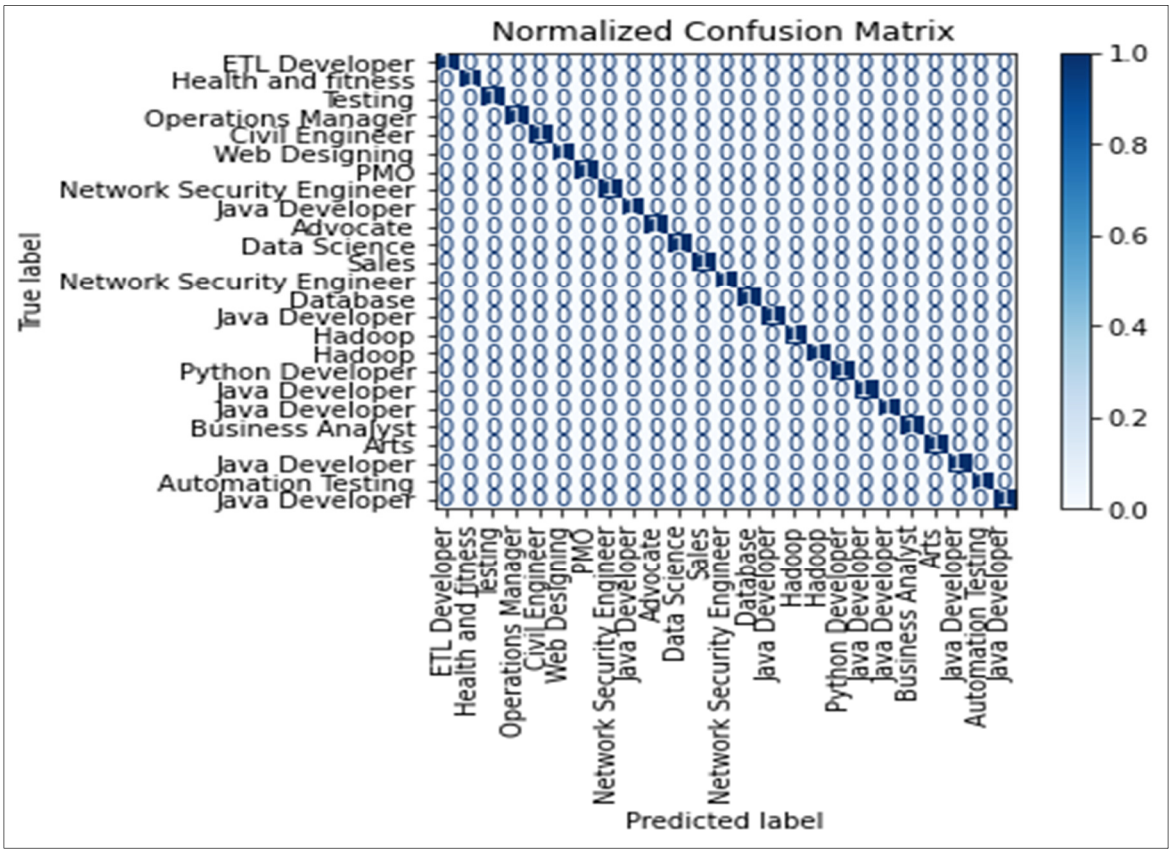


Fig. 9: Normalized Confusion Matrix of Actual vs Predicted class labels

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

accuracy however, fails to perform well on the test or *unseen data* [33]. The *overfitting* problem yields *higher train accuracy* and *lower test accuracy*. Whereas, the *underfitting* problem occurs when the model fails to perform well either on test or train data. The *underfitting* yields slighter lower accuracies for train and test data.

It is evident from Fig. 8 that the proposed models in this study are neither *overfitting* nor *underfitting* the train or test data. The trained models equally perform better on training and test data. It can be inferred that the overall process of Natural Language Processing (NLP) and Machine Learning (ML) techniques is employed efficiently to yield balanced and better performance on test and train data.

Fig. 9 illustrates the normalized Confusion Matrix of actual versus predicted class categories of best performing Support Vector Machine (SVM) – Linear SVC classifier. Since, the classifier yielded over 98% true class prediction accuracy and it is depicted in normalized confusion matrix in Fig. 9. The predicted values for confusion matrix are automatically normalized in plot confusion matrix library of sklearn metrics in Python implementation. The results shown in Fig. 9 are evident that the employed NLP techniques and well-trained classification model for Resumes categorization truly predicted categories.

5. LIMITATIONS AND FUTURE WORK

The major limitation and challenge for the Resume Classification and Recommendation task is finding an appropriate and standard dataset to process using the NLP techniques and train the ML models. Since the resume is not a standard document and there is no specific industry standard, thus major efforts were put on processing the documents in the dataset which were parsed from different formats and layouts. Moreover, the dataset size was a bit low to train the ML model for generalized classification. However, efforts were put to find a more suitable dataset for the classification task. The study achieved significant accuracy and performance gain on Resume Classification in different job categories. Therefore, in future work, the model will be extended to match the content of the resume with the provided job description. The extension in future work will enable the proposed system suitable for the complete recruiting process. The proposed system will perform the most tedious tasks of recruiting process; categorization and recommendation of suitable resumes for a given job description.

6. CONCLUSION

Resume classification is a time-consuming, costly, and tedious job for an organization. In this regard, this study proposes an automated approach that uses various machine learning and NLP techniques for the classification of Resumes. The proposed methodology used several NLP and ML techniques for preprocessing data, feature extraction and representation, model construction, and evaluation for the Resume Classification task. The study results suggested that the TF-IDF vectorizer performed best in feature extraction and representation as the extracted features yielded excellent results on almost all classifiers. However, the Support Vector Machine (SVM) class algorithms such as (Linear, SVC, NuSVC, and SGD) performed exceptionally good with over 98% and 96% accuracy respectively on the train and unseen test data. The study results are quite encouraging to automate the job application categorization and recommendation based on the content of the Resumes. The developed system can be deployed in real-time settings for an employer to automate the recruiting process.

REFERENCES

1. Koyande B.A., Walke R.S., Jondhale M.G., "Predictive Human Resource Candidate Ranking System", *International Journal of Research in Engineering, Science and Management*, Vol.3, No.1, January 2020.
2. Al-Otaibi S.T., Ykhlef M., A., "Survey of job recommender systems", *International Journal of Physical Sciences*, Vol. 7, No. 29, pp. 5127-5142, 2012.
3. Färber F., Weitzel T., Keim T., *"An automated recommendation approach to selection in*

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

personnel recruitment", Proceedings of the 9th American Conference on Information Systems (AMCIS), Association of Information Systems Electronic Library, pp. 2329-2339, 2003.

4. Breaugh J.A., "The use of biodata for employee selection: Past research and future directions", *Human Resource Management Review*, Vol. 19, No.3, pp. 219-231, 2009.
5. Lin, Y., Lei H., Addo P.C., Li X., "Machine learned resume-job matching solution", arXiv preprint arXiv:1607.07657, 2016.
6. Yi X., Allan J., Croft W.B.. "Matching resumes and jobs based on relevance models", *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 809-810, Amsterdam, The Netherlands, July 2007.
7. Sebastiani F., "Machine learning in automated text categorization", ACM Computing Surveys Vol. 34, No.1, pp. 1-47, 2002.
8. Nigam K., Mccallum A.K., Thrun S., Mitchell T., "Text classification from labeled and unlabeled documents using EM", *Machine Learning*, Vol. 39, No. 2-3, pp. 103-134, 2000.
9. Uysal A.K., Gunal S., "The impact of preprocessing on text classification. Information", *Processing and Management*, Vol. 50, No.1, pp. 104-112, 2014.
10. Otter D.W., Medina J.R., Kalita J.K., "A Survey of the Usages of Deep Learning for Natural Language Processing", *IEEE Transactions on Neural Networks and Learning Systems*, Vol. 32, No.2, pp. 604-624. February 2021.
11. Parkhe V., Biswas B., "*Sentiment analysis of movie reviews: finding most important movie aspects using driving factors", Soft Computing*, Vol. 20, No. 9, pp. 3373-3379. 2016.
12. Bakshi R.K., Kaur N., Kaur R., Kaur G., "Opinion mining and sentiment analysis", *Proceedings of the 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*, pp. 452-455, New Delhi, India, 16-18 March 2016.
13. Sivapalan S., Sadeghian A., Rahnama H., Madni A.M., "Recommender systems in e-commerce", *Proceedings of the World Automation Congress (WAC)*, pp. 179-184, Kona, Hawaii, 2014.
14. Srifi M., Oussous A., Lahcen A.A., Mouline S., "Recommender Systems Based on Collaborative Filtering Using Review Texts—A Survey", Information, Vol. 11, No. 6, pp. 317, 2020.
15. Mujtaba G., Shuib L., Raj R.G., Majeed N., Al-Garadi M.A., "Email classification research trends: review and open issues", *IEEE Access*, Vol. 5, p. 9044-9064, 2017.
16. Theofilatos K., Likothanassis S., Karathanasopoulos A., "Modeling and trading the EUR/USD exchange rate using machine learning techniques", *Engineering, Technology and Applied Science Research*, Vol.2, No. 5, pp. 269-272. 2012.
17. Rahman A. Khan M.N.A., "A Classification Based Model to Assess Customer Behavior in Banking Sector", *Engineering, Technology and Applied Science Research*, Vol. 8, No.3, pp. 2949-2953. 2018.
18. Al-Garadi M.A., Khan M.S., Varathan K.D., Mujtaba G., Al-Kabsi A.M., "Using online social networks to track a pandemic: A systematic review", *Journal of Biomedical Informatics*, Vol. 62, pp. 1-11, 2016.
19. Mujtaba G., Shuib L., Raj R.G., Rajandram R., Shaikh K., "Automatic text classification of ICD-10 related CoD from complex and free text forensic autopsy reports", *Proceedings of the 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*. Anaheim, C.A., U.S.A., pp. 1055-1058, 18 – 20 December 2016.
20. Gonzalez T., Santos P., Orozco F., Alcaraz M., Zaldivar V., De Obeso A., Garcia A., "Adaptive Employee Profile Classification for Resource Planning Tool", *Proceedings of the Annual SRII Global Conference*, pp. 544-553, San Jose, C.A., U.S.A., 24-27 July 2012.
21. Guo S., Alamudun F., Hammond T., "RésuMatcher: A personalized résumé-job matching system", *Expert Systems with Applications*, Vol. 60, pp. 169-182, 2016.
22. Golec A., Kahya E., "A fuzzy model for competency-based employee evaluation and selection", *Computers and Industrial Engineering*, Vol. 52, No.1, pp. 143-161, 2007.
23. Gopalakrishna S.T., Vijayraghavan V.,

Downloaded from search.informit.org/doi/10.3316/informit.263278216314684. on 09/10/2026 05:56 PM AEST; UTC+10:00. © Mehran University Research Journal Of Engineering & Technology , 2022.
Available under a Creative Commons Attribution Licence.

"Automated Tool for Resume Classification Using Sementic Analysis", *International Journal of Artificial Intelligence and Applications,* Vol. 10, No.1, 2019.

24. Sayfullina L., Malmi E., Liao Y., Jung A., "Domain adaptation for resume classification using convolutional neural networks", *Proceedings of the International Conference on Analysis of Images, Social Networks and Texts*, pp. 82-93, Springer, 2017.
25. Ramos J., "Using TF-IDF to determine word relevance in document queries", *Proceedings of the First Instructional Conference on Machine Learning*, New Jersey, U.S.A., 2003.
26. Xu J., "*An extended one-versus-rest support vector machine for multi-label classification", Neurocomputing*, Vol. 74, No. 17, pp. 3114-3124, 2011.
27. Loper E., Bird S., "NLTK: the natural language to olkit", *arXiv preprint cs/0205028*, 2002
28. Kibriya A.M., Frank E., Pfahringer B., Holms G., "*Multinomial naive bayes for text categorization revisited*, in *Australasian Joint Conference on Artificial Intelligence*", Springer, 2004.
29. McCallum A., Nigam K., "A comparison of event models for naive bayes text classification", *Proceedings of the AAAI-98 Workshop on Learning for Text Categorization*, pp. 41-48, 1998.
30. Raschka S., "Naive bayes and text classification introduction and theory", arXiv preprint arXiv:1410.5329, 2014.
31. Xu S., "Bayesian Naïve Bayes classifiers to text classification", *Journal of Information Science*, Vol. 44, No.1, pp. 48-59, 2018.
32. Schölkopf B., Smola A.J., Bach F., "*Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*", MIT Press, 2002.
33. Suykens J.A., Vandewalle J., "Least Squares Support Vector Machine Classifiers", *Neural Processing Letters*, Vol. 9, No.3, pp. 293-300, 1999.